PROPUESTA TÉCNICA Y FUNCIONAL

Sistema de Fidelización Custom

Puntos · Sellos · Compras recurrentes · Niveles · Canje validado

Configurable para cualquier rubro



7 de 10 compras · ¡A la 10ª, tu beneficio!

Stack: Laravel (PHP) + JavaScript con formularios AJAX

Octubre 2026 · Versión 1.0

Índice

1. Resumen ejecutivo . . . 4

2. Objetivos y alcance . . . 4

2.1 Objetivos . . . 4

2.2 Dentro del alcance . . . 4

2.3 Fuera del alcance inicial . . . 5

3. Actores y roles . . . 5

4. Modelo multi-negocio y personalización por rubro . . . 5

4.1 Configuración por negocio . . . 5

5. Motor de reglas (núcleo del sistema) . . . 6

5.1 Tipos de regla . . . 6

5.2 Condiciones combinables . . . 6

5.3 Ejemplo de regla almacenada . . . 7

5.4 Diseño técnico: patrón Strategy . . . 7

5.5 Plantillas por rubro . . . 8

6. Beneficios, cupones y validación del canje . . . 8

6.1 Catálogo de recompensas . . . 8

6.2 Cupón con código único y QR . . . 8

6.3 Validación en caja (paso a paso) . . . 8

7. Ledger de puntos (libro de movimientos) . . . 9

8. Identificación y registro del cliente . . . 10

9. Modelo de datos . . . 10

10. Arquitectura técnica . . . 11

10.1 Stack recomendado . . . 11

10.2 Organización del código . . . 12

11. Patrón AJAX en el frontend . . . 12

11.1 Convención de respuesta JSON . . . 12

11.2 Helper reutilizable ajaxForm() . . . 13

11.3 Uso en la pantalla del cajero . . . 13

11.4 Controlador correspondiente . . . 13

11.5 Otras piezas AJAX . . . 14

12. Pantallas del sistema . . . 14

12.1 Panel de administración . . . 14

12.2 Pantalla del cajero (optimizada para tablet) . . . 15

12.3 Portal del cliente (web responsive / PWA) . . . 15

13. Flujos clave . . . 15

13.1 Registro de compra y acumulación . . . 15
13.2 Canje de puntos por recompensa . . . 15
13.3 Tareas programadas . . . 16
14. Seguridad y antifraude . . . 16
15. API para integraciones (POS / e-commerce) . . . 16
16. Roadmap por fases . . . 17
17. Estructura de carpetas propuesta . . . 17
18. Ejemplos de configuración por rubro . . . 18
19. Estrategia de pruebas . . . 19
20. Próximos pasos y decisiones abiertas . . . 19
Plan de arranque sugerido . . . 20

1. Resumen ejecutivo

Se propone construir una **plataforma de fidelización de clientes genérica y configurable**, capaz de adaptarse a cualquier rubro (cafeterías, restaurantes, retail, gimnasios, peluquerías, farmacias, talleres, etc.) sin reprogramar el sistema. La clave del diseño es un **motor de reglas**: en lugar de codificar "1 punto por cada $1.000" o "el 10º café gratis", el administrador del negocio **configura** esas reglas desde un panel y el sistema las evalúa automáticamente en cada compra.

El sistema permite acumular **puntos**, completar **tarjetas de sellos**, premiar la **compra recurrente** (X compras en N días), desbloquear beneficios al alcanzar un **monto acumulado**, subir de **nivel** (Bronce, Plata, Oro) y **canjear beneficios** mediante cupones con código único y QR que el cajero **valida** en el punto de venta.

Tecnológicamente se construye sobre **Laravel** (backend, base de datos, reglas de negocio, API) y **JavaScript** en el frontend, donde **todos los formularios se envían por AJAX**: registrar una compra, crear una regla o validar un cupón entrega respuesta inmediata sin recargar la página.

Aspecto	Propuesta
Público objetivo	Negocios de cualquier rubro que quieran retener clientes y aumentar la recompra.
Diferenciador	Reglas configurables + validación de canjes segura + multi-negocio.
Backend	Laravel 12, MySQL/MariaDB, colas y tareas programadas.
Frontend	Blade como plantilla base + JS (fetch/Axios) para formularios y tablas AJAX.
Usuarios	Super Admin, Admin de negocio, Cajero/Operador, Cliente final.
Entrega	Por fases: MVP funcional primero, luego niveles, portal cliente, API e integraciones.

2. Objetivos y alcance

2.1 Objetivos

- Aumentar la **frecuencia de compra** y el **ticket promedio** de los clientes de cada negocio.
- Permitir que cada negocio diseñe su propio programa sin depender de un programador.
- Registrar compras y validar canjes en segundos desde el mostrador (PC, tablet o celular).
- Entregar métricas claras: clientes activos, recurrencia, puntos emitidos vs. canjeados, beneficios más usados.
- Garantizar trazabilidad total: cada punto ganado o gastado queda registrado y auditado.

2.2 Dentro del alcance

- Gestión de negocios, sucursales, usuarios y roles.
- Registro de clientes y su identificación (teléfono, email, documento, QR personal).
- Registro de compras (manual por cajero o vía API) y evaluación automática de reglas.
- Puntos, sellos, umbrales de monto, frecuencia, multiplicadores, niveles, bienvenida, cumpleaños y referidos.
- Catálogo de recompensas, emisión de cupones, validación y canje con protección contra doble uso.

- Vencimiento de puntos y cupones, reversas por compras anuladas, ajustes manuales auditados.
- Dashboard y reportes; portal del cliente; notificaciones por email.

2.3 Fuera del alcance inicial

- Procesamiento de pagos (el sistema registra la compra, no la cobra).
- Aplicación móvil nativa (se cubre con web responsive / PWA; app nativa puede ser fase futura).
- Facturación electrónica e inventario (se pueden integrar vía API si el negocio ya los tiene).

3. Actores y roles

Rol	Qué puede hacer
Super Admin (dueño de la plataforma)	Crea y suspende negocios, define planes/límites, ve métricas globales, soporte.
Admin de negocio	Configura marca, sucursales, usuarios, reglas, niveles, recompensas y campañas. Ve reportes. Hace ajustes manuales (auditados).
Cajero / Operador	Busca o registra clientes, registra compras, ve puntos ganados al instante, valida y canjea cupones. Sin acceso a configuración.
Cliente final	Consulta saldo, progreso de sellos, nivel, cupones disponibles e historial; muestra su QR; canjea puntos por recompensas.

Los permisos se implementan con roles y permisos granulares (paquete *spatie/laravel-permission*), de modo que un negocio pueda crear roles intermedios (por ejemplo, "Supervisor de sucursal" que puede anular compras pero no editar reglas).

4. Modelo multi-negocio y personalización por rubro

Para que el sistema sirva a **cualquier rubro** y eventualmente pueda ofrecerse como servicio (SaaS), se recomienda un modelo **multi-tenant de base de datos única**: todas las tablas de negocio tienen la columna `business_id` y un *Global Scope* de Eloquent filtra automáticamente los datos del negocio del usuario autenticado. Así un negocio nunca ve datos de otro.

> **¿Y si solo lo usará un negocio?** El mismo diseño funciona con un único registro en `businesses`. Agregar multi-negocio desde el inicio cuesta poco y evita una migración dolorosa después.

4.1 Configuración por negocio

Parámetro	Ejemplo	Para qué sirve
Rubro	Cafetería, Gimnasio, Retail…	Precargar plantillas de reglas sugeridas.
Nombre de la unidad	Puntos / Estrellas / Sellos / Coins	La interfaz habla el idioma del negocio.
Moneda y formato	CLP, $1.000 / USD, $10.00	Cálculos y visualización correctos.
Logo y colores	#4F46E5	Portal de cliente y cupones con su marca.

Parámetro	Ejemplo	Para qué sirve
Identificador de cliente	Teléfono / RUT-DNI / Email	Cómo busca el cajero al cliente.
Vencimiento de puntos	12 meses, nunca	Política de caducidad.
Zona horaria	America/Santiago	Reglas por día/horario correctas.

5. Motor de reglas (núcleo del sistema)

El motor de reglas es lo que hace al sistema **custom**. Cada regla se guarda en la tabla `loyalty_rules` con tres partes: un **tipo**, unas **condiciones** (cuándo aplica) y una **recompensa** (qué entrega). Las condiciones y recompensas se guardan en columnas JSON, lo que permite agregar nuevas opciones sin modificar la base de datos.

5.1 Tipos de regla

Tipo	Descripción	Ejemplo
points_per_amount	Puntos proporcionales al monto. Configurable: cada cuánto, redondeo, monto mínimo, tope por compra.	1 punto por cada $1.000. Compra de $15.800 = 15 pts.
fixed_per_purchase	Puntos fijos por cada compra/visita sin importar el monto.	10 pts por visita al gimnasio.
stamp_card	Tarjeta de sellos: cada compra válida suma un sello; al completar N, se emite un cupón y la tarjeta se reinicia.	A la 10ª compra, café gratis.
spend_threshold	Al acumular un monto total en un periodo (o histórico) se desbloquea un beneficio.	Gasta $100.000 en el mes y recibe 15% dcto.
frequency	Compras recurrentes: X compras dentro de N días (ventana móvil o calendario).	4 visitas en 30 días = sesión gratis.
multiplier	Multiplica los puntos de otras reglas según día, horario, categoría, sucursal o campaña.	Martes x2. Categoría "Postres" x1,5.
birthday	Bono o cupón en el día/mes de cumpleaños.	Postre gratis en tu mes.
welcome	Bono por registrarse o por primera compra.	50 pts de bienvenida.
referral	Premio para quien refiere y para el referido cuando este compra por primera vez.	100 pts para cada uno.
tier	Niveles por puntos o gasto en una ventana (ej. últimos 12 meses), con beneficios propios.	Oro: x1,5 puntos y envío gratis.

5.2 Condiciones combinables

Cualquier regla puede restringirse con condiciones opcionales; todas deben cumplirse (lógica AND):

- **Vigencia**: fecha de inicio y fin (campañas temporales).
- **Días y horarios**: solo lunes a jueves, o de 15:00 a 18:00 (horas valle).
- **Sucursales**: aplica solo en ciertas sucursales.
- **Monto mínimo** de la compra o **categorías** de producto incluidas/excluidas.
- **Nivel del cliente**: solo para clientes Plata u Oro.

- **Límites de uso**: máximo por cliente (1 vez, N veces por mes) y tope global (primeros 100 clientes).
- **Prioridad y exclusividad**: orden de evaluación y si la regla es acumulable con otras.

5.3 Ejemplo de regla almacenada

loyalty_rules — registro de ejemplo (JSON)

```
{
  "name": "10ª compra: café gratis",
  "type": "stamp_card",
  "is_active": true,
  "priority": 10,
  "conditions": {
    "min_amount": 2000,
    "branches": [1, 3],
    "days_of_week": [1, 2, 3, 4, 5],
    "valid_from": "2026-11-01", "valid_until": null
  },
  "config": { "stamps_required": 10, "reset_on_complete": true },
  "reward": { "type": "coupon", "reward_id": 7, "expires_in_days": 30 }
}
```

5.4 Diseño técnico: patrón Strategy

Cada tipo de regla es una clase PHP que implementa una interfaz común. El **RuleEngine** recibe la compra, carga las reglas activas del negocio, filtra las que cumplen condiciones y delega en la estrategia correspondiente. Agregar un nuevo tipo de regla en el futuro = crear una nueva clase y registrarla, sin tocar el resto del código.

app/Loyalty/RuleEngine.php (simplificado)

```
interface LoyaltyRuleStrategy
{
    public function supports(string $type): bool;
    public function apply(LoyaltyRule $rule, Purchase $purchase, RuleContext $ctx): RuleResult;
}

final class RuleEngine
{
    /** @param iterable<LoyaltyRuleStrategy> $strategies */
    public function __construct(private iterable $strategies, private ConditionChecker $conditions) {}

    public function evaluate(Purchase $purchase): EvaluationResult
    {
        $ctx    = RuleContext::for($purchase);           // cliente, nivel, historial, fecha
        $result = new EvaluationResult();

        $rules = LoyaltyRule::active()->orderBy('priority')->get();
        foreach ($rules as $rule) {
            if (! $this->conditions->passes($rule, $purchase, $ctx)) continue;
            $strategy = $this->resolve($rule->type);
            $result->merge($strategy->apply($rule, $purchase, $ctx));
            if ($rule->is_exclusive) break;
        }
        return $result->applyMultipliers();   // los multiplicadores se aplican al final
    }
}
```

5.5 Plantillas por rubro

Al crear un negocio y elegir su rubro, el sistema ofrece **plantillas de reglas** listas para activar (ver sección 18). El admin puede usarlas tal cual o ajustarlas, lo que reduce la curva de aprendizaje.

6. Beneficios, cupones y validación del canje

6.1 Catálogo de recompensas

Tipo de recompensa	Ejemplo	Cómo se obtiene
Producto/servicio gratis	Café, lavado, corte de pelo	Canje por puntos o desbloqueo por regla (sellos, frecuencia).
Descuento porcentual	15% en la próxima compra	Canje por puntos, nivel o umbral de monto.
Descuento de monto fijo	$5.000 de descuento	Canje por puntos.
Regalo / upgrade	Tamaño grande sin costo	Cumpleaños, campañas.
Beneficio de nivel	Envío gratis permanente	Automático al pertenecer al nivel.

Cada recompensa define: costo en puntos (si aplica), stock opcional, sucursales donde se puede usar, días de validez del cupón generado, monto mínimo de compra para usarlo y límite por cliente.

6.2 Cupón con código único y QR

Al canjear puntos o completar una regla, se crea un registro en `redemptions` con un **código único no adivinable** (por ejemplo `CAF-7K3Q-9XZP`, generado con caracteres aleatorios criptográficamente seguros) y un **QR** que contiene un token firmado. El cliente lo muestra desde su portal o lo recibe por email.

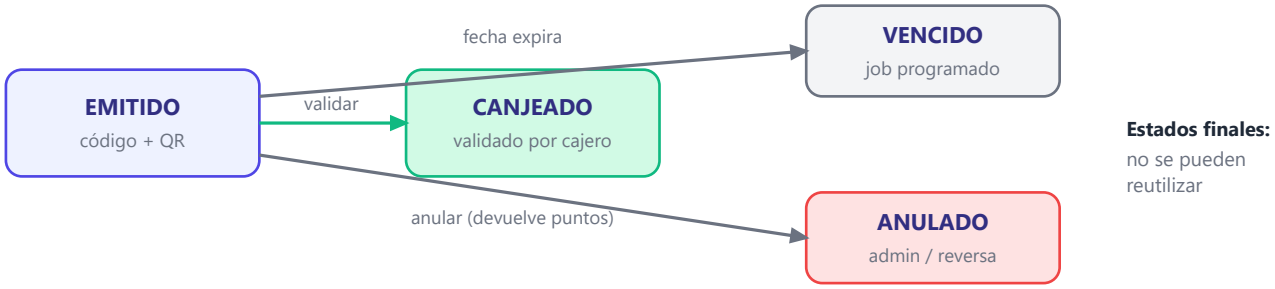


Figura 3. Ciclo de vida de un cupón / canje.

6.3 Validación en caja (paso a paso)

- El cajero abre **Validar canje** y escanea el QR con la cámara (librería JS *html5-qrcode*) o escribe el código.
- Se envía un AJAX a `POST /redemptions/validate`: el servidor responde con el detalle del beneficio, el cliente y si es válido (vigente, de este negocio, sucursal permitida, no usado).
- El cajero confirma con **Canjear**: AJAX a `POST /redemptions/{id}/redeem`.
- El servidor, dentro de una **transacción con bloqueo de fila** (`lockForUpdate`), verifica nuevamente el estado y lo marca como CANJEADO con fecha, sucursal y cajero.

- Si dos cajeros intentan canjear el mismo cupón al mismo tiempo, solo uno lo logra; el otro recibe "Este cupón ya fue utilizado el 17/10 a las 14:32 en Sucursal Centro".

app/Services/RedemptionService.php

```
public function redeem(Redemption $redemption, User $cashier): Redemption
{
    return DB::transaction(function () use ($redemption, $cashier) {
        $r = Redemption::whereKey($redemption->id)->lockForUpdate()->firstOrFail();

        if ($r->status !== RedemptionStatus::Issued) {
            throw new RedemptionException("Cupón {$r->status->label()}");
        }
        if ($r->expires_at?->isPast()) {
            throw new RedemptionException('Cupón vencido');
        }

        $r->update([
            'status'      => RedemptionStatus::Redeemed,
            'redeemed_at' => now(),
            'redeemed_by' => $cashier->id,
            'branch_id'   => $cashier->current_branch_id,
        ]);
        event(new RewardRedeemed($r));
        return $r;
    });
}
```

7. Ledger de puntos (libro de movimientos)

Los puntos **nunca se editan directamente**. Cada cambio de saldo es un movimiento inmutable en `point_transactions`, como un libro contable. Esto da trazabilidad total, permite auditorías, reversas limpias y reportes exactos.

Tipo	Signo	Origen	Ejemplo
earn	+	Compra / regla	+15 pts por compra #1042
bonus	+	Bienvenida, cumpleaños, referido, campaña	+50 pts de bienvenida
redeem	−	Canje de recompensa	−100 pts por cupón CAF-7K3Q
expire	−	Job de vencimiento	−30 pts vencidos (lote oct-2025)
reverse	−/+	Anulación de compra o cupón	−15 pts por anulación compra #1042
adjust	−/+	Ajuste manual del admin (con motivo obligatorio)	+20 pts compensación reclamo

- **Saldo** = suma de movimientos; se guarda además en `customers.points_balance` (actualizado en la misma transacción) para consultas rápidas.
- **Vencimiento FIFO**: cada movimiento *earn* guarda `expires_at` y `remaining`; los canjes consumen primero los puntos más antiguos. Un job diario vence lo que corresponda y avisa al cliente 7 días antes.
- **Reversas**: anular una compra genera movimientos inversos y retrocede el progreso de sellos/frecuencia asociado.

8. Identificación y registro del cliente

- **Búsqueda en caja** por teléfono, email, documento (RUT/DNI) o escaneando el **QR personal** del cliente. El buscador es AJAX con autocompletado (consulta a partir del 3er carácter, con *debounce*).
- **Alta rápida**: si no existe, el cajero lo registra en un modal con nombre + teléfono (y opcional email y cumpleaños) sin salir de la pantalla de compra.
- **Portal del cliente**: ingreso sin contraseña mediante código de 6 dígitos (OTP) enviado al email (o SMS/WhatsApp en fase 3). Muestra saldo, sellos, nivel, cupones y su QR.
- **Consentimiento**: aceptación de términos y de comunicaciones de marketing por separado (protección de datos personales).
- **Un cliente, varios negocios**: en modo multi-negocio, el mismo teléfono puede ser cliente de varios negocios, cada uno con su saldo independiente.

9. Modelo de datos

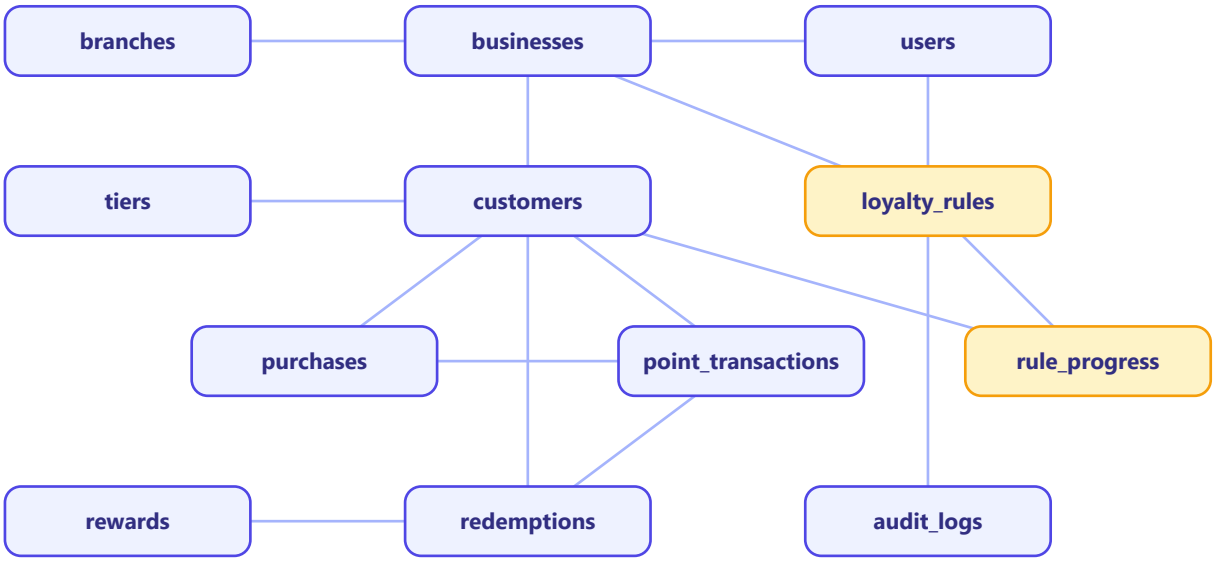


Figura 4. Modelo entidad-relación simplificado (todas las tablas llevan business_id).

Tabla	Campos principales	Notas
businesses	id, name, slug, industry, currency, points_label, logo, colors, settings(json), status	Un registro por negocio.
branches	id, business_id, name, address, is_active	Sucursales.
users	id, business_id, branch_id, name, email, password	Admins y cajeros (roles vía spatie).
customers	id, business_id, name, phone, email, document, birthdate, qr_token, points_balance, tier_id, total_spent, last_purchase_at, marketing_opt_in	Índices únicos por (business_id, phone) etc.
purchases	id, business_id, branch_id, customer_id, user_id, amount, external_ref, items(json), status, purchased_at	external_ref evita duplicados desde POS.

Tabla	Campos principales	Notas
loyalty_rules	id, business_id, name, type, conditions(json), config(json), reward(json), priority, is_exclusive, is_active, usage_limit_*	Corazón del motor.
rule_progress	id, customer_id, loyalty_rule_id, counter, amount_acc, window_start, completed_count	Avance de sellos, frecuencia y umbrales.
point_transactions	id, business_id, customer_id, type, points, remaining, expires_at, source_type, source_id, rule_id, note, created_by	Ledger inmutable (polimórfico al origen).
rewards	id, business_id, name, type, points_cost, value, stock, valid_days, conditions(json), image, is_active	Catálogo.
redemptions	id, business_id, customer_id, reward_id, code, token, status, issued_at, expires_at, redeemed_at, redeemed_by, branch_id	Cupones y canjes.
tiers	id, business_id, name, min_points / min_spent, window_months, benefits(json), sort	Niveles.
campaigns	id, business_id, name, starts_at, ends_at, rule_ids, audience(json)	Fase 2-3.
audit_logs	id, business_id, user_id, action, auditable_type/id, old(json), new(json), ip	Ajustes, anulaciones, cambios de reglas.

10. Arquitectura técnica

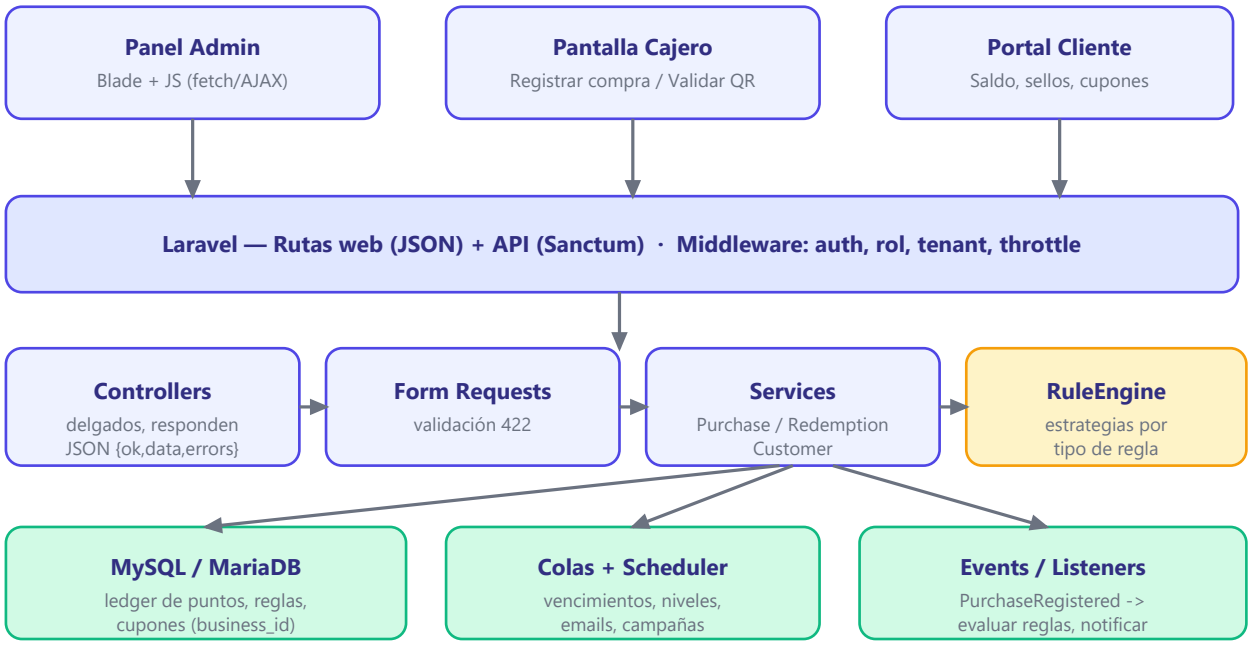


Figura 1. Arquitectura general por capas.

10.1 Stack recomendado

Capa	Tecnología	Motivo
Framework	Laravel 12 (PHP 8.2+)	Compatible con el PHP 8.2 instalado (XAMPP). Laravel 13 requiere PHP 8.3; se puede migrar luego.
Base de datos	MySQL 8 / MariaDB 10.6+	Soporte JSON, transacciones y bloqueos de fila.

Capa	Tecnología	Motivo
Frontend	Blade + JavaScript (fetch o Axios), Vite	Páginas servidas por Laravel; interacción 100% AJAX.
UI	Bootstrap 5 (o Tailwind CSS)	Componentes listos: modales, toasts, tablas responsive.
Reactividad ligera (opcional)	Alpine.js	Mostrar/ocultar, contadores, formularios dinámicos de reglas sin un framework pesado.
Autenticación	Laravel Breeze + Sanctum	Login de staff; tokens para API y portal cliente.
Roles/permisos	spatie/laravel-permission	Roles por negocio y permisos granulares.
QR	simplesoftwareio/simple-qrcode + html5-qrcode (JS)	Generar QR en servidor y escanearlos con cámara.
Gráficos	Chart.js	Dashboard.
Colas/tareas	Queue (database/redis) + Scheduler	Vencimientos, niveles, emails sin bloquear al usuario.
Pruebas	Pest / PHPUnit	Cobertura del motor de reglas y endpoints.

10.2 Organización del código

- **Controllers delgados**: reciben la petición, delegan y devuelven JSON.
- **Form Requests**: toda la validación; Laravel responde automáticamente 422 con los errores por campo.
- **Services**: lógica de negocio (*PurchaseService*, *RedemptionService*, *CustomerService*, *PointsLedger*).
- **Loyalty/**: *RuleEngine*, *ConditionChecker* y una clase por estrategia.
- **Events/Listeners**: *PurchaseRegistered*, *RewardUnlocked*, *TierChanged*, *RewardRedeemed* → emails, auditoría, webhooks.
- **API Resources**: formato consistente de las respuestas JSON.
- **Policies**: quién puede ver/editar qué; combinadas con el scope de negocio.

11. Patrón AJAX en el frontend

Todos los formularios del sistema siguen un mismo patrón para que la experiencia sea fluida y el código sea reutilizable: **un único helper JavaScript** intercepta el envío, manda los datos por *fetch*, muestra un indicador de carga, pinta los errores de validación bajo cada campo y muestra un aviso (toast) con el resultado.

11.1 Convención de respuesta JSON

```
// Éxito (200/201)
{ "ok": true, "message": "Compra registrada", "data": { "points_earned": 15,
  "balance": 230, "stamps": { "current": 8, "required": 10 }, "unlocked": [] } }

// Error de validación (422) — formato nativo de Laravel
{ "message": "El monto es obligatorio.", "errors": { "amount": ["El monto es obligatorio."] } }

// Error de negocio (409 / 400)
{ "ok": false, "message": "Este cupón ya fue utilizado." }
```

11.2 Helper reutilizable ajaxForm()

Frontend — envío de formularios sin recargar

```
// resources/js/ajax-form.js
const csrf = document.querySelector('meta[name="csrf-token"]').content;

export function ajaxForm(form, { onSuccess } = {}) {
  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    clearErrors(form);
    const btn = form.querySelector('[type=submit]');
    btn.disabled = true; btn.classList.add('is-loading');

    try {
      const res = await fetch(form.action, {
        method: form.method || 'POST',
        headers: { 'X-CSRF-TOKEN': csrf, 'Accept': 'application/json' },
        body: new FormData(form),
      });
      const json = await res.json();

      if (res.status === 422) return showErrors(form, json.errors);
      if (!res.ok) return toast(json.message || 'Error inesperado', 'error');

      toast(json.message, 'success');
      onSuccess?.(json.data, form);
    } catch {
      toast('Sin conexión con el servidor', 'error');
    } finally {
      btn.disabled = false; btn.classList.remove('is-loading');
    }
  });
}

function showErrors(form, errors) {
  for (const [field, msgs] of Object.entries(errors)) {
    const input = form.querySelector(`[name="${field}"]`);
    input?.classList.add('is-invalid');
    input?.insertAdjacentHTML('afterend', `<div class="invalid-feedback">${msgs[0]}</div>`);
  }
}
```

11.3 Uso en la pantalla del cajero

```
ajaxForm(document.querySelector('#form-compra'), {
  onSuccess(data, form) {
    form.reset();
    renderCustomerCard(data);           // actualiza saldo, sellos y nivel en pantalla
    if (data.unlocked.length) showRewardModal(data.unlocked);   // "¡Desbloqueó un café gratis!"
    purchasesTable.reload();            // refresca la tabla de últimas compras por AJAX
  }
});
```

11.4 Controlador correspondiente

```
class PurchaseController extends Controller
{
    public function store(StorePurchaseRequest $request, PurchaseService $service)
    {
        $result = $service->register($request->validated(), $request->user());

        return response()->json([
            'ok'      => true,
            'message' => "Compra registrada: +{$result->pointsEarned} {$result->label}",
            'data'    => new PurchaseResultResource($result),
        ], 201);
    }
}
```

11.5 Otras piezas AJAX

- **Tablas** (clientes, compras, canjes) con búsqueda, filtros y paginación del lado servidor vía AJAX.
- **Modales** que cargan formularios parciales (crear cliente, editar regla) y se cierran al guardar.
- **Constructor de reglas dinámico**: al elegir el tipo de regla se muestran solo los campos pertinentes.
- **Acciones en línea**: activar/desactivar regla o recompensa con un interruptor que hace PATCH al instante.
- **Confirmaciones** para acciones destructivas (anular compra, ajustar puntos) con motivo obligatorio.

12. Pantallas del sistema

12.1 Panel de administración

Pantalla	Contenido
Dashboard	KPIs: clientes activos (30 días), nuevos clientes, compras, ticket promedio, tasa de recompra, puntos emitidos vs. canjeados, cupones pendientes; gráficos de tendencia; top clientes; recompensas más canjeadas.
Reglas	Listado con estado, prioridad y uso. Constructor por pasos: tipo → configuración → condiciones → recompensa → vista previa ("Con esta regla, una compra de $15.000 un martes daría 30 pts").
Recompensas	Catálogo con imagen, costo, stock, vigencia y estadísticas de canje.
Niveles	Definición de niveles, umbrales y beneficios.
Clientes	Buscador, filtros (nivel, inactivos, cumpleaños del mes), ficha con historial de compras, movimientos de puntos, cupones y ajuste manual.
Compras	Historial con filtros por sucursal/cajero/fecha; anulación con reversa automática.
Sucursales y usuarios	Alta de sucursales, cajeros, asignación de roles.
Reportes	Exportación a Excel/CSV: movimientos, canjes, clientes, rendimiento por regla.
Configuración	Marca, unidad de puntos, moneda, vencimientos, plantillas de email.

12.2 Pantalla del cajero (optimizada para tablet)

- **Registrar compra**: buscador de cliente grande → tarjeta del cliente (saldo, sellos, nivel, cupones disponibles) → campo de monto → botón "Registrar". Resultado inmediato con animación de puntos y aviso si desbloqueó un beneficio.
- **Validar canje**: botón de cámara para leer QR o campo de código; muestra el beneficio y botón "Canjear".
- **Canjear puntos**: desde la tarjeta del cliente, elegir una recompensa del catálogo y generar el cupón al momento.
- **Últimas operaciones** del turno, con opción de solicitar anulación.

12.3 Portal del cliente (web responsive / PWA)

- Saldo de puntos y fecha del próximo vencimiento.
- Progreso visual de tarjetas de sellos y metas de frecuencia ("Te faltan 2 visitas este mes").
- Nivel actual y cuánto falta para el siguiente.
- Cupones disponibles con su QR, catálogo de recompensas canjeables e historial.
- Código QR personal para identificarse en caja; enlace de referidos.

13. Flujos clave

13.1 Registro de compra y acumulación



Todo ocurre sin recargar la página: el formulario se limpia y el panel del cliente se actualiza al instante.

Figura 2. Flujo de registro de compra y acumulación.

Internamente, *PurchaseService::register()* ejecuta todo en una transacción: crea la compra (rechazando duplicados por `external_ref` o por compra idéntica en los últimos 60 segundos), llama al *RuleEngine*, escribe los movimientos en el ledger, actualiza `rule_progress`, emite cupones desbloqueados, recalcula el nivel y dispara el evento *PurchaseRegistered* (emails y webhooks van a la cola).

13.2 Canje de puntos por recompensa

- Cliente o cajero elige la recompensa → AJAX `POST /customers/{id}/redeem-points`.
- Servidor verifica saldo suficiente, stock y límites dentro de una transacción con bloqueo sobre el cliente.
- Registra movimiento *redeem* (consumo FIFO), crea el cupón EMITIDO y responde con código + QR.
- El cupón se usa en ese momento o después; su validación sigue el flujo de la sección 6.3.

13.3 Tareas programadas

Tarea	Frecuencia	Acción
ExpirePoints	Diaria 03:00	Vence puntos con expires_at pasado y notifica 7 días antes.
ExpireCoupons	Diaria 03:15	Marca cupones vencidos.
RecalculateTiers	Diaria	Baja/sube niveles según ventana móvil.
BirthdayRewards	Diaria 08:00	Emite bonos de cumpleaños.
InactiveCustomers	Semanal	Lista clientes inactivos para campañas de reactivación ("Te extrañamos").

14. Seguridad y antifraude

Riesgo	Medida
Cajero registra compras falsas para un conocido	Tope de monto y de compras por cliente/día configurable; alertas de patrones; auditoría por cajero; anulación solo por supervisor.
Doble canje de un cupón	Transacción + lockForUpdate + estado único; token QR firmado (HMAC).
Adivinar códigos de cupón	Códigos aleatorios largos con alfabeto sin caracteres ambiguos; rate limiting en validación.
Compra duplicada (doble clic / reintento POS)	Botón deshabilitado durante el envío; external_ref única; clave de idempotencia en API.
Un negocio ve datos de otro	Global scope por business_id + Policies + tests automáticos de aislamiento.
Ajustes manuales abusivos	Motivo obligatorio, permiso específico, registro en audit_logs con valores antes/después.
Ataques web comunes	CSRF, validación server-side, escapado en Blade, consultas Eloquent parametrizadas, HTTPS, contraseñas con bcrypt/argon.
Datos personales	Consentimiento explícito, exportación/eliminación de datos del cliente a solicitud.

15. API para integraciones (POS / e-commerce)

Para negocios que ya tienen un sistema de ventas, se expone una API REST autenticada con tokens de Sanctum por negocio. Así cada venta del POS o de la tienda online suma puntos automáticamente sin que el cajero la digite.

Método	Endpoint	Descripción
POST	/api/v1/purchases	Registra una compra (idempotente por external_ref) y devuelve puntos y beneficios.
POST	/api/v1/purchases/{ref}/void	Anula una compra y revierte puntos.
GET	/api/v1/customers/lookup?phone=	Busca cliente por teléfono/email/documento.
POST	/api/v1/customers	Crea cliente.
GET	/api/v1/customers/{id}/balance	Saldo, nivel, sellos y cupones disponibles.
POST	/api/v1/redemptions/validate	Valida un código de cupón.

Método	Endpoint	Descripción
POST	/api/v1/redemptions/{code}/redeem	Canjea un cupón.

Webhooks opcionales (fase 3): el sistema notifica a una URL del negocio eventos como *reward.unlocked*, *tier.changed* o *redemption.redeemed*, firmados con un secreto compartido.

16. Roadmap por fases

Fase	Contenido	Resultado
Fase 0 Base	Instalación Laravel, auth, roles, multi-negocio, layout, helper AJAX, sucursales y usuarios.	Esqueleto funcionando con login y panel vacío.
Fase 1 MVP	Clientes (búsqueda/alta rápida), registro de compras, ledger, reglas *points_per_amount*, *fixed_per_purchase* y *stamp_card*, recompensas, canje por puntos, cupones con código/QR y validación, anulaciones, dashboard básico.	Un negocio real puede operar su programa de puntos y sellos.
Fase 2 Fidelización avanzada	Reglas *spend_threshold*, *frequency*, *multiplier*, *birthday*, *welcome*; niveles; condiciones avanzadas; vencimientos; portal del cliente con OTP; emails; reportes y exportaciones.	Programa completo y autogestionable por el cliente.
Fase 3 Integración y crecimiento	API pública + webhooks, referidos, campañas segmentadas, WhatsApp/SMS, Apple/Google Wallet, PWA instalable, panel Super Admin con planes.	Producto listo para ofrecerse como SaaS.

Recomendación: construir y probar el MVP con un negocio real (piloto) antes de la fase 2. Las reglas que realmente se usan suelen ser pocas; el piloto indica qué priorizar.

17. Estructura de carpetas propuesta

```
app/
├── Enums/                RuleType, RedemptionStatus, TransactionType
├── Events/               PurchaseRegistered, RewardUnlocked, RewardRedeemed, TierChanged
├── Listeners/            SendRewardEmail, WriteAuditLog, DispatchWebhook
├── Http/
│   ├── Controllers/
│   │   ├── Admin/        DashboardController, RuleController, RewardController,
│   │   │                 CustomerController, TierController, ReportController
│   │   ├── Cashier/      PurchaseController, RedemptionController, CustomerLookupController
│   │   ├── Portal/       PortalController, OtpLoginController
│   │   └── Api/V1/       PurchaseApiController, CustomerApiController, RedemptionApiController
│   ├── Requests/         StorePurchaseRequest, StoreRuleRequest, RedeemRequest ...
│   ├── Resources/        CustomerResource, PurchaseResultResource ...
│   └── Middleware/       SetCurrentBusiness, EnsureBusinessIsActive
├── Loyalty/
│   ├── RuleEngine.php
│   ├── ConditionChecker.php
│   ├── RuleContext.php  /  RuleResult.php  /  EvaluationResult.php
│   └── Strategies/       PointsPerAmount, FixedPerPurchase, StampCard, SpendThreshold,
│                         Frequency, Multiplier, Birthday, Welcome, Referral
├── Models/               Business, Branch, Customer, Purchase, LoyaltyRule, RuleProgress,
│                         PointTransaction, Reward, Redemption, Tier, AuditLog
│   └── Scopes/BusinessScope.php
├── Policies/
├── Services/             PurchaseService, RedemptionService, PointsLedger, TierService
└── Console/Commands/     ExpirePoints, ExpireCoupons, RecalculateTiers, BirthdayRewards
resources/
├── views/                layouts/, admin/, cashier/, portal/, components/
└── js/                   app.js, ajax-form.js, toast.js, tables.js, rule-builder.js, qr-scanner.js
database/
├── migrations/
└── seeders/              DemoBusinessSeeder, IndustryTemplatesSeeder
tests/
├── Unit/Loyalty/         un test por estrategia + ConditionChecker
└── Feature/              Purchase, Redemption (concurrencia), TenantIsolation, Api
```

18. Ejemplos de configuración por rubro

La misma plataforma, configurada distinto. Estas combinaciones se ofrecerán como **plantillas** al crear el negocio:

Rubro	Reglas sugeridas	Recompensas
Cafetería	stamp_card: 1 sello por compra ≥ $2.000; al 10º sello cupón. multiplier: x2 sellos lunes 15–18 h. birthday.	Café gratis, pastel gratis en cumpleaños.
Retail / tienda	points_per_amount: 1 pt c/ $1.000. tier: Plata ≥ $300.000/año (x1,25), Oro ≥ $800.000/año (x1,5). welcome 100 pts.	Descuentos de $5.000 / $10.000 por puntos; envío gratis nivel Oro.
Gimnasio	fixed_per_purchase: 10 pts por asistencia. frequency: 12 asistencias en 30 días → bono. referral.	Clase personal gratis, mes con 20% dcto., merchandising.
Peluquería / estética	stamp_card: 5 visitas → 6º servicio gratis. spend_threshold: $150.000 en 3 meses → tratamiento.	Corte gratis, tratamiento capilar.
Restaurante	points_per_amount + multiplier x3 martes y miércoles (días bajos). birthday: postre. frequency: 4 visitas/mes.	Postre, bebida, 15% en la cuenta.
Lavado de autos / taller	stamp_card: 8 lavados → 1 gratis. frequency recordatorio cada 30 días.	Lavado gratis, revisión sin costo.
Farmacia / minimarket	points_per_amount con categorías excluidas (medicamentos con receta). multiplier por categoría.	Cupones de monto fijo.

19. Estrategia de pruebas

- **Unitarias del motor de reglas**: por cada estrategia, casos de borde (monto justo en el límite, redondeo, regla fuera de horario, límite de usos alcanzado, multiplicadores combinados, reinicio de tarjeta de sellos).
- **Feature tests de endpoints AJAX**: respuestas 201/422/409, formato JSON, permisos por rol.
- **Concurrencia**: dos canjes simultáneos del mismo cupón → solo uno exitoso.
- **Aislamiento multi-negocio**: un usuario del negocio A no puede leer ni modificar datos del negocio B.
- **Ledger**: el saldo cacheado siempre coincide con la suma de movimientos (test + comando de verificación).
- **Seeders de demo**: negocio de ejemplo por rubro con clientes y compras ficticias para probar y presentar.

tests/Unit/Loyalty/StampCardTest.php (Pest)

```
it('entrega un cupón al completar la tarjeta de sellos', function () {
    $rule = LoyaltyRule::factory()->stampCard(required: 3)->create();
    $customer = Customer::factory()->create();

    purchase($customer, 2500);
    purchase($customer, 2500);
    $result = purchase($customer, 2500);

    expect($result->unlocked)->toHaveCount(1)
        ->and($customer->redemptions()->issued()->count())->toBe(1)
        ->and(RuleProgress::for($customer, $rule)->counter)->toBe(0);  // tarjeta reiniciada
});
```

20. Próximos pasos y decisiones abiertas

Antes de comenzar la implementación conviene cerrar estos puntos (la propuesta ya incluye un valor por defecto recomendado para cada uno):

Decisión	Opciones	Recomendado
¿Un negocio o varios?	Uso propio de un negocio / plataforma para varios negocios	Multi-negocio desde el inicio (bajo costo, evita migraciones).
Identificador principal del cliente	Teléfono / documento / email	Teléfono + QR personal.
Integración con POS existente	Registro manual / API	Manual en MVP; API en fase 3.
Framework CSS	Bootstrap 5 / Tailwind	Bootstrap 5 (más rápido para paneles con modales y tablas).
Canal de notificaciones	Email / WhatsApp / SMS	Email en fase 2; WhatsApp en fase 3.
Hosting	VPS (Laravel Forge/Ploi) / hosting compartido	VPS pequeño con colas y cron habilitados.
Versión de PHP	Mantener 8.2 / actualizar a 8.3+	Laravel 12 sobre 8.2 ahora; actualizar PHP antes de producción.

Plan de arranque sugerido

- 1. Validar esta propuesta y cerrar las decisiones abiertas.
- 2. Crear el proyecto Laravel en `F:\sistemaFidelizacionCustom`, configurar BD (MySQL de XAMPP), auth y roles (Fase 0).
- 3. Implementar el modelo de datos, el ledger y el RuleEngine con sus tests.
- 4. Construir la pantalla del cajero (registrar compra + validar canje) con el helper AJAX.
- 5. Panel de administración de reglas y recompensas; dashboard básico.
- 6. Piloto con un negocio real, ajustar y continuar con la Fase 2.